Concurrency Controls

MCQ Question Bank | OAVS TGT Computer Teacher Examination

Syllabus Covered (10.1)
1. Problems of Concurrent Transactions
2. Locks
3. Time-Stamps
4. Optimistic Scheduling
5. MVT

Questions are strictly limited to these topics. MVT = Multiversion Timestamp Ordering.

Section	Questions	Q Range
1. Problems of Concurrent Transactions	18	Q1-Q18
2. Locks and Lock Compatibility	18	Q19-Q36
3. Timestamp-Based Concurrency Control	17	Q37-Q53
4. Optimistic Scheduling	15	Q54-Q68
5. MVT (Multiversion Timestamp Ordering)	17	Q69-Q85
6. Transaction Schedule-Based Questions	16	Q86-Q101
7. Comparisons and Tricky Questions	17	Q102-Q118
8. Mixed OAVS TGT Practice Set	15	Q119-Q133
Total	**133**	

How to use: Attempt each question, then check the green answer strip and the yellow explanation box. Notation: R-ts / W-ts = read / write timestamp of an item; TS(T) = timestamp of transaction T; S = shared lock; X = exclusive lock.

Section 1: Problems of Concurrent Transactions

Q1 | Lost Update | Concurrency Controls

In a DBMS, a 'lost update' occurs when:

A. two transactions update the same item from the same old value and one overwrites the other's result

B. a committed update is erased from the database after the transaction ends normally

C. a transaction reads data written by another transaction that later aborts and is rolled back

D. a transaction sees different values when it reads the same item twice during its execution

Ans: A

Explanation: Both transactions read the same old value, so the later write overwrites the earlier one. The earlier update is lost (W-W conflict).

Q2 | Dirty Read | Concurrency Controls

Reading a data item written by another transaction that has not yet committed is called:

A. Lost update

B. Lock conversion

C. Dirty read

D. Phantom read

Ans: C

Explanation: Uncommitted (dirty) data may later be rolled back, so the reader may use a value that never officially existed. It is a W-R conflict.

Q3 | Incorrect Summary | Concurrency Controls

T2 totals accounts X and Y while T1 transfers money from X to Y. T2 reads X after T1 debits it but reads Y before T1 credits it, so the total is wrong. This problem is called:

A. Phantom read (extra rows appear)

B. Lost update (write-write overwrite)

C. Unrepeatable read (item changed)

D. Incorrect summary (inconsistent retrieval)

Ans: D

Explanation: An aggregate computed during another transaction's updates sees some items before and some after the change, giving an inconsistent total.

Q4 | Unrepeatable Read | Concurrency Controls

T1 reads item A twice. Between the two reads, T2 updates A and commits, so T1 gets two different values. This is:

A. Incorrect summary

B. Lost update

C. Unrepeatable read

D. Dirty read

Ans: C

Explanation: The same item returns different values within one transaction because another transaction changed it in between (R-W conflict).

Q5 | Phantom Read | Concurrency Controls

T1 runs the same condition-based query twice. Between the runs, T2 inserts a new qualifying row and commits. The second result has an extra row. This is:

A. Phantom read

B. Dirty read

C. Lost update

D. Unrepeatable read

Ans: A

Explanation: A phantom is a new row that appears for the same predicate. Unrepeatable read concerns a changed value of an existing row.

Q6 | Lost Update (Numeric) | Concurrency Controls

A = 100. T1 reads A; T2 reads A. T1 writes A = A - 10; then T2 writes A = A + 20 and commits. What is the final value of A, and which problem occurs?

A. 110; lost update

B. 90; dirty read

C. 120; lost update

D. 120; dirty read

Ans: C

Explanation: T2 computed from the old value 100 and overwrote T1's write. A serial run would give 110, so T1's update is lost.

Q7 | Dirty Read (Scenario) | Concurrency Controls

T1 changes A from 300 to 500 but has not committed. T2 reads 500. T1 then aborts and A returns to 300. The value 500 used by T2:

A. was protected by a shared lock held by T1

B. never existed in any committed database state

C. was committed but later lost in a crash

D. was valid in some serial schedule of T1 and T2

Ans: B

Explanation: T2 acted on uncommitted data that was rolled back. Dirty reads can also force cascading rollbacks.

Q8 | Basic Concepts | Concurrency Controls

Which of the following is NOT a problem of concurrent transactions?

A. Incorrect summary

B. Dirty read

C. Serializable schedule

D. Lost update

Ans: C

Explanation: A serializable schedule is the desired outcome of concurrency control, not a problem.

Q9 | ACID - Isolation | Concurrency Controls

The ACID property that concurrency control mechanisms mainly enforce is:

A. Durability

B. Isolation

C. Consistency

D. Atomicity

Ans: B

Explanation: Isolation means concurrent transactions behave as though executed one at a time, which is what concurrency control provides.

Q10 | Need for Concurrency | Concurrency Controls

The main reason for allowing concurrent execution of transactions is to:

A. remove the need for isolation between transactions running together

B. improve throughput and resource utilisation by overlapping CPU and I/O work

C. guarantee that no transaction ever aborts because of other transactions

D. reduce the physical storage size of the database on disk

Ans: B

Explanation: Interleaving keeps CPU and disks busy and reduces waiting time for short transactions, but it needs control to stay correct.

Q11 | Conflict Pairs | Concurrency Controls

Which pair of operations by different transactions on the same item can lead to a dirty read?

A. Read followed by Write later (R-W)

B. Write followed by Write again (W-W)

C. Write followed by Read before commit (W-R)

D. Read followed by Read by both (R-R)

Ans: C

Explanation: A dirty read is a read of another transaction's uncommitted write. R-W gives unrepeatable read; W-W gives lost update.

Q12 | Cascading Rollback | Concurrency Controls

T2 reads a value written by T1, and T3 reads a value written by T2. T1 then aborts, so T2 and T3 must also be rolled back. This is called:

A. Phantom read

B. Cascading rollback

C. Lost update

D. Unrepeatable read

Ans: B

Explanation: Dirty reads create dependencies, so the abort of one transaction forces the abort of all transactions that read its uncommitted data.

Q13 | Recoverable Schedule | Concurrency Controls

A schedule is recoverable if:

A. every transaction commits before any other transaction is allowed to start

B. all transactions perform all of their writes before any of their reads

C. no two transactions ever access the same data item at the same time

D. every transaction commits only after all transactions whose data it read have committed

Ans: D

Explanation: Recoverability ensures a committed transaction never depends on data that is later rolled back.

Q14 | Cascadeless Schedule | Concurrency Controls

In a cascadeless schedule, a transaction may read an item written by another transaction only after that transaction has:

A. written a second item

B. started

C. read the same item

D. committed

Ans: D

Explanation: Reading only committed data removes dirty reads, so no cascading rollback can happen. Every cascadeless schedule is recoverable.

Q15 | Serializability | Concurrency Controls

A schedule is conflict serializable if it is conflict equivalent to:

A. some serial schedule of the same transactions

B. any other non-serial schedule

C. a recoverable schedule of the same transactions

D. a cascadeless schedule of the same transactions

Ans: A

Explanation: Swapping non-conflicting operations must be able to turn the schedule into a serial one.

Q16 | Schedule Definition | Concurrency Controls

In concurrency control, a schedule is:

A. the plan used by the query optimiser to execute a single query

B. the chronological order in which operations of concurrent transactions execute

C. the set of all timestamps that are assigned to every data item

D. the list of all locks currently held by the active transactions

Ans: B

Explanation: A schedule fixes the interleaving order of read/write operations of several transactions. It is serial if there is no interleaving.

Q17 | Serial Schedule | Concurrency Controls

In a serial schedule:

A. every transaction is rolled back and then restarted once

B. only read operations of the transactions are allowed to run

C. the operations of all transactions are interleaved arbitrarily

D. the operations of different transactions are never interleaved

Ans: D

Explanation: Each transaction runs completely before the next one begins. Serial schedules are always correct but give poor throughput.

Q18 | Concurrency Control | Concurrency Controls

Concurrent execution of transactions is considered correct if the resulting schedule is:

A. serial only

B. serializable

C. free of all read operations

D. interleaved at the operation level

Ans: B

Explanation: Serializability (equivalence to some serial schedule) is the correctness criterion. Requiring only serial schedules is unnecessarily strict.

Section 2: Locks and Lock Compatibility

Q19 | Lock Concept | Concurrency Controls

In concurrency control, a lock is best described as:

A. a variable on a data item that controls how concurrent transactions access it

B. a backup copy of a data item that is kept on disk for recovery use

C. an index structure that speeds up the search for a particular data item

D. a log record that is written to stable storage when a transaction commits

Ans: A

Explanation: A lock restricts access to a data item so that conflicting operations cannot interleave improperly.

Q20 | Shared Lock | Concurrency Controls

A transaction holding a shared (S) lock on a data item can:

A. only read it, while other transactions may also hold S locks on it

B. read and write it, while all other access to the item is blocked

C. delete it from the table but not read its value at all

D. only write it, while all other readers of the item are blocked

Ans: A

Explanation: S (read) locks are shareable, since reads do not conflict with reads.

Q21 | Exclusive Lock | Concurrency Controls

An exclusive (X) lock on a data item allows the holder to:

A. write it only if other transactions still hold S locks on it

B. read it only, blocking just the other writers of the item

C. both read and write it, with no other lock held on it

D. only read it, sharing the item with any number of other readers

Ans: C

Explanation: X (write) locks give sole access: they conflict with both S and X requests.

Q22 | Lock Compatibility | Concurrency Controls

Which combination of locks held by two different transactions on the same item is compatible?

A. S and X

B. S and S

C. X and S

D. X and X

Ans: B

Explanation: Only S-S is compatible. Any combination involving an X lock conflicts.

Q23 | Lock Request | Concurrency Controls

T1 holds an S-lock on item Q. T2 requests an X-lock on Q. The lock manager will:

A. grant T2 and make T1 wait
B. grant T2 immediately and abort T1
C. make T2 wait until T1 releases its lock
D. grant T2 and let T1 continue reading

Ans: C

Explanation: S and X are incompatible, so the later requester T2 waits until the conflicting lock is released.

Q24 | Two-Phase Locking | Concurrency Controls

The two phases of the Two-Phase Locking (2PL) protocol are:

A. growing phase (acquire locks) followed by shrinking phase (release locks)
B. validation phase (conflict check) followed by commit phase (log flush)
C. locking phase (S and X locks) followed by timestamp phase (ordering)
D. read phase (local copies) followed by write phase (database update)

Ans: A

Explanation: In 2PL a transaction first only acquires locks, then only releases them.

Q25 | 2PL Rule | Concurrency Controls

In basic 2PL, after a transaction releases its first lock, it:

A. cannot acquire any new lock
B. can acquire any lock after waiting
C. can acquire only shared locks
D. must commit immediately

Ans: A

Explanation: Releasing a lock starts the shrinking phase, in which no new lock may be requested.

Q26 | 2PL Guarantee | Concurrency Controls

Basic 2PL guarantees:

A. schedules free of cascading rollbacks
B. conflict serializable schedules
C. that only serial schedules are produced
D. that no transaction is ever rolled back

Ans: B

Explanation: 2PL ensures serializability but not cascadeless schedules. Strict 2PL is needed for that.

Q27 | Strict 2PL | Concurrency Controls

In strict 2PL, a transaction holds all its _____ locks until it commits or aborts.

A. no
B. binary
C. shared
D. exclusive

Ans: D

Explanation: Keeping X locks until the end stops others from reading uncommitted writes, so cascading rollbacks are avoided.

Q28 | Lock Point | Concurrency Controls

The lock point of a transaction in 2PL is the:

A. moment at which it first starts waiting for a lock
B. moment at which it requests its first lock
C. point at which the transaction obtains its final lock
D. moment at which it finally commits

Ans: C

Explanation: The lock point ends the growing phase. Ordering transactions by lock points gives an equivalent serial order.

Q29 | Lock Upgrade | Concurrency Controls

Converting an S-lock held by a transaction on an item into an X-lock is called lock:

A. inheritance
B. upgrade
C. downgrade
D. release

Ans: B

Explanation: Upgrade is S to X (allowed only in the growing phase of 2PL); downgrade is X to S.

Q30 | Binary Lock | Concurrency Controls

A binary lock on a data item has how many possible states?

A. One: locked only
B. Two: locked and unlocked
C. Three: shared, exclusive and unlocked
D. Four: read, write, shared and exclusive

Ans: B

Explanation: A binary lock is simply locked or unlocked. S/X locks are multiple-mode locks.

Q31 | Lock Granularity | Concurrency Controls

Choosing a finer lock granularity (e.g., a single record instead of a whole table) generally:

A. decreases both concurrency and locking overhead

B. has no effect on concurrency but increases overhead

C. increases concurrency and also reduces locking overhead

D. increases concurrency but increases locking overhead

Ans: D

Explanation: Smaller locked units allow more parallel work, but more locks must be managed.

Q32 | Lock Conversion in 2PL | Concurrency Controls

In basic 2PL, lock upgrades and lock downgrades are allowed respectively in:

A. the growing phase only and the shrinking phase only

B. the shrinking phase only, for both operations

C. the growing phase only, for both operations

D. the shrinking phase only and the growing phase only

Ans: A

Explanation: Upgrade (S to X) acquires stronger rights, so it belongs to the growing phase. Downgrade (X to S) is a release, so it belongs to the shrinking phase.

Q33 | Lock Manager | Concurrency Controls

The lock table maintained by the lock manager records:

A. which transactions hold or wait for locks on each data item, and in which mode

B. the read timestamp and the write timestamp of every data item, along with the owner transaction

C. the validation result of every active transaction, with its start and finish times

D. the old versions of every data item that exists in the database, with their creators

Ans: A

Explanation: The lock table maps each locked item to its holders, requested modes and the queue of waiting transactions.

Q34 | Conservative 2PL | Concurrency Controls

In conservative (static) 2PL, a transaction:

A. releases all of its locks before it starts executing, and then runs

B. acquires its locks only at the time of commit, not before it

C. acquires all the locks it needs before it begins execution

D. acquires only shared locks throughout its execution, never exclusive ones

Ans: C

Explanation: Conservative 2PL pre-declares its read and write sets and locks everything first, so it has no growing phase during execution.

Q35 | Strict 2PL Schedules | Concurrency Controls

Schedules produced by strict 2PL are always:

A. conflict serializable and cascadeless

B. conflict serializable but may have cascading rollbacks

C. non-serializable but recoverable

D. serial schedules only

Ans: A

Explanation: Strict 2PL keeps 2PL's serializability and holds X-locks until commit, so no transaction reads uncommitted data.

Q36 | Lock Queue | Concurrency Controls

T1 holds an X-lock on A. T2 and T3 request S-locks on A. When T1 releases its lock:

A. only T2 can be granted, and T3 must wait for T2

B. both T2 and T3 can be granted S-locks together

C. neither T2 nor T3 can ever obtain the lock

D. T2 and T3 must both take X-locks instead

Ans: B

Explanation: S-S is compatible, so both waiting readers can share A once the conflicting X-lock is gone.

Section 3: Timestamp-Based Concurrency Control

Q37 | Time-Stamp | Concurrency Controls

In timestamp-based concurrency control, a timestamp is:

A. the total number of data items that a transaction has accessed so far during its execution

B. a lock mode that is granted to a transaction when an item is read by it, for later reuse

C. the clock time at which the last full database backup was taken, kept for later recovery

D. a unique value given to a transaction at its start, used to order transactions

Ans: D

Explanation: Timestamps (system clock or logical counter) fix a serial order: older transaction = smaller timestamp.

Q38 | TS Ordering | Concurrency Controls

In timestamp ordering, if TS(Ti) < TS(Tj), the protocol ensures the result is equivalent to a serial schedule in which:

A. the order is chosen randomly at commit time

B. Ti executes before Tj

C. Ti and Tj run in parallel with no fixed order

D. Tj executes before Ti

Ans: B

Explanation: The serializability order is the timestamp order.

Q39 | W-timestamp | Concurrency Controls

W-timestamp(Q) denotes:

A. the largest timestamp of any transaction that successfully executed write(Q)

B. the smallest timestamp of any transaction that has ever successfully written Q

C. the system time at which the data item Q was first created in the database

D. the timestamp of the transaction that most recently read the item Q successfully

Ans: A

Explanation: R-ts(Q) and W-ts(Q) keep the largest timestamps of successful readers and writers of Q.

Q40 | Read Rule | Concurrency Controls

Under basic timestamp ordering, T issues read(Q) with TS(T) < W-ts(Q). The result is:

A. the read is allowed and W-ts(Q) is set to TS(T)

B. T waits until Q is unlocked

C. the read is rejected and T is rolled back

D. the read is allowed and R-ts(Q) is updated

Ans: C

Explanation: T needs a value of Q that a younger transaction has already overwritten, so T must be rolled back.

Q41 | Write Rule | Concurrency Controls

Under basic timestamp ordering, T issues write(Q) with TS(T) < R-ts(Q). The result is:

A. the write is allowed and W-ts(Q) = TS(T)

B. the write is ignored (Thomas write rule)

C. T waits for the younger reader to commit

D. the write is rejected and T is rolled back

Ans: D

Explanation: A younger transaction has already read Q, so T's write would be too late. The Thomas rule applies only when TS(T) < W-ts(Q).

Q42 | Thomas Write Rule | Concurrency Controls

The Thomas write rule can allow schedules that are:

A. conflict serializable but never view serializable

B. view serializable but not conflict serializable

C. strictly serial schedules only

D. neither view serializable nor conflict serializable

Ans: B

Explanation: Ignoring obsolete writes admits more schedules than basic TO, which allows only conflict serializable ones.

Q43 | TS Scenario - Read | Concurrency Controls

TS(T) = 10, R-ts(Q) = 5, W-ts(Q) = 12. T issues read(Q). Outcome under basic TO?

A. Allowed; R-ts(Q) becomes 10

B. Allowed; W-ts(Q) becomes 10

C. Rejected; T is rolled back

D. Allowed; R-ts(Q) remains 5

Ans: C

Explanation: TS(T) = 10 < W-ts(Q) = 12, so a younger transaction already wrote Q; the read is rejected.

Q44 | TS Scenario - Write | Concurrency Controls

TS(T) = 20, R-ts(Q) = 15, W-ts(Q) = 10. T issues write(Q). Outcome under basic TO?

A. Rejected because TS(T) > R-ts(Q)

B. Allowed; W-ts(Q) becomes 20

C. Ignored because W-ts(Q) < TS(T)

D. Allowed; R-ts(Q) becomes 20

Ans: B

Explanation: TS(T) is not less than R-ts(Q) or W-ts(Q), so the write is executed and W-ts(Q) is set to TS(T).

Q45 | TS Scenario - Thomas | Concurrency Controls

TS(T) = 8, R-ts(Q) = 6, W-ts(Q) = 10. T issues write(Q) under the Thomas write rule. Outcome?

A. The write is performed and W-ts(Q) becomes 8

B. R-ts(Q) becomes 8 and T waits

C. T is rolled back

D. The write is ignored and T continues

Ans: D

Explanation: TS(T) = 8 > R-ts = 6, so no reader is affected. TS(T) < W-ts = 10, so the write is obsolete and skipped (basic TO would roll back).

Q46 | No-Lock Protocol | Concurrency Controls

Which statement is true of the timestamp-ordering protocol?

A. It uses a separate read, validation and write phase before every commit

B. It uses shared and exclusive locks on every item that each transaction accesses

C. It does not use locks; conflicts are resolved by rolling back transactions

D. It makes younger transactions wait for older ones until they unlock all items

Ans: C

Explanation: Transactions never wait for locks in TO. A transaction that arrives too late is rolled back and restarted.

Q47 | Restart | Concurrency Controls

A transaction rolled back by the timestamp-ordering protocol is restarted with:

A. a new, larger timestamp

B. the timestamp of the transaction that caused the rollback

C. timestamp 0

D. its old timestamp

Ans: A

Explanation: Restarting with the old timestamp would cause the same conflict again.

Q48 | R-ts Update | Concurrency Controls

After T successfully executes read(Q), R-ts(Q) is set to:

A. min(R-ts(Q), TS(T))

B. W-ts(Q)

C. always TS(T)

D. max(R-ts(Q), TS(T))

Ans: D

Explanation: R-ts(Q) must always hold the largest timestamp of any successful reader, so it never decreases.

Q49 | TS Scenario - Read (R-ts) | Concurrency Controls

TS(T) = 12, R-ts(Q) = 15, W-ts(Q) = 10. T issues read(Q). Outcome under basic TO?

A. Allowed; R-ts(Q) stays 15

B. Allowed; R-ts(Q) becomes 12

C. Allowed; W-ts(Q) becomes 12

D. Rejected because 12 < R-ts(Q)

Ans: A

Explanation: A read is checked only against W-ts(Q): 12 >= 10, so it is allowed. R-ts(Q) = max(15, 12) = 15.

Q50 | Rollback Effect | Concurrency Controls

When T is rolled back under basic timestamp ordering, transactions that have read values written by T:

A. must also be rolled back (cascading rollback)

B. are only delayed until T restarts

C. are converted into read-only transactions

D. continue unaffected because timestamps protect them

Ans: A

Explanation: Basic TO does not guarantee cascadeless schedules, so dependent transactions must also be aborted.

Q51 | Timestamp Generation | Concurrency Controls

Timestamps for transactions can be generated using:

A. the size of the lock table

B. the largest data item value in the database

C. the system clock or a logical counter

D. random numbers generated for every operation

Ans: C

Explanation: A clock value or a counter incremented per transaction gives unique, increasing values.

Q52 | Write Condition | Concurrency Controls

Under basic timestamp ordering, write(Q) by T is executed when:

A. TS(T) < R-ts(Q) and TS(T) < W-ts(Q)

B. TS(T) >= R-ts(Q) and TS(T) < W-ts(Q)

C. TS(T) < R-ts(Q) and TS(T) >= W-ts(Q)

D. TS(T) >= R-ts(Q) and TS(T) >= W-ts(Q)

Ans: D

Explanation: A younger reader or writer has not yet touched Q only if TS(T) is at least both timestamps.

Q53 | TS Sequence | Concurrency Controls

TS(T1) = 15 and TS(T2) = 12. Initially R-ts(Q) = W-ts(Q) = 10. T1 executes write(Q), then T2 executes write(Q). Under basic TO, T2's write:

A. is rejected and T2 is rolled back

B. waits until T1 commits

C. succeeds and R-ts(Q) becomes 12

D. succeeds and W-ts(Q) becomes 12

Ans: A

Explanation: After T1's write W-ts(Q) = 15. T2's TS = 12 < 15, so its write is too late and T2 is rolled back (Thomas rule would ignore it).

Section 4: Optimistic Scheduling

Q54 | Optimistic Basics | Concurrency Controls

Optimistic concurrency control is based on the assumption that:

A. conflicts among transactions are rare, so checking can be deferred until the end

B. every transaction conflicts with others, so all items must be locked first

C. all transactions are read-only

D. all transactions must run serially

Ans: A

Explanation: Transactions run without locks and are checked only at the end, which pays off when conflicts are rare.

Q55 | Three Phases | Concurrency Controls

The correct order of phases in optimistic scheduling is:

A. Read, then Write, then Validation

B. Write, then Validation, then Read

C. Read, then Validation, then Write

D. Validation, then Read, then Write

Ans: C

Explanation: Work is done on local copies (read), checked (validation), and applied to the database (write).

Q56 | Read Phase | Concurrency Controls

In the read phase of optimistic scheduling, a transaction:

A. assigns fresh timestamps to all of the data items that it touches

B. waits for an exclusive lock on every item before it may read it

C. reads committed values and keeps its updates in private local copies

D. writes directly to the database after locking each item it updates

Ans: C

Explanation: Nothing in the database is changed during the read phase, so aborting is cheap.

Q57 | Validation Phase | Concurrency Controls

The validation phase of optimistic scheduling checks whether:

A. the transaction has the largest timestamp among all of the active transactions

B. the transaction's updates would break serializability with other transactions

C. the log records of the transaction have been flushed to stable disk storage

D. all of the locks that were held by other transactions have been released

Ans: B

Explanation: Read/write sets are compared with those of concurrent transactions to decide if the transaction can safely commit.

Q58 | Write Phase | Concurrency Controls

In optimistic scheduling, local updates are applied to the database:

A. only if the transaction holds an X lock on the item
B. during the read phase, as each update is made
C. only after the transaction passes validation
D. before the validation phase has even begun

Ans: C

Explanation: Only the write phase makes changes permanent, and it runs only after successful validation.

Q59 | Validation Failure | Concurrency Controls

If a transaction fails validation in optimistic scheduling, it is:

A. allowed to commit anyway after a short wait
B. made to wait for an X lock on the conflicting item
C. converted into a read-only transaction and finished
D. aborted and restarted, discarding its local copies

Ans: D

Explanation: No database changes were made, so abort just discards the local workspace.

Q60 | Suitability | Concurrency Controls

Optimistic scheduling is most suitable when:

A. long update transactions compete for the same records
B. many transactions update the same hot items
C. conflicts are frequent and aborts are expensive
D. most transactions are read-only or conflicts are rare

Ans: D

Explanation: Low contention means few validation failures, so lock overhead is saved at little cost.

Q61 | Validation Timestamps | Concurrency Controls

In the validation-based protocol, each transaction T has which three timestamps?

A. Read(T), Lock(T) and Unlock(T)
B. R-ts(T), W-ts(T) and TS(T)
C. Start(T), Validation(T) and Finish(T)
D. Begin(T), Commit(T) and Abort(T)

Ans: C

Explanation: Start marks the read phase beginning, Validation marks entry to validation, Finish marks end of the write phase.

Q62 | Serial Order | Concurrency Controls

In the validation-based protocol, the serializability order of transactions is determined by:

A. the Validation(T) timestamp

B. the Start(T) timestamp

C. the order in which locks are granted

D. the Finish(T) timestamp

Ans: A

Explanation: The protocol uses Validation(T) as the transaction's timestamp for the equivalent serial order.

Q63 | Drawback | Concurrency Controls

The main drawback of optimistic scheduling under heavy contention is:

A. many transactions fail validation, causing repeated aborts

B. older versions of every data item must be stored for readers

C. transactions must wait for locks for very long periods of time

D. timestamps overflow quickly and must be reassigned to all items

Ans: A

Explanation: Conflicts are found only at the end, after all the work has been done, so the work is lost on abort.

Q64 | Other Name | Concurrency Controls

Optimistic concurrency control is also known as:

A. pessimistic scheduling approach (lock first, then run)

B. two-phase locking approach with a lock point

C. lock-based approach (shared and exclusive locks)

D. validation-based (certification) approach

Ans: D

Explanation: It is called validation- or certification-based because conflict checking happens at the validation step.

Q65 | Validation Scenario | Concurrency Controls

Ti: Start = 1, Validation = 5, Finish = 9 and Tj: Start = 4, Validation = 11. Write-set(Ti) = {X}, read-set(Tj) = {Y}. Tj's validation against Ti:

A. fails because Ti and Tj overlap in time during their execution phases

B. succeeds, since Finish(Ti) lies between Start(Tj) and Validation(Tj) and the sets do not overlap

C. succeeds only if Ti is rolled back first and then restarted again

D. fails because Finish(Ti) is greater than Start(Tj), so they overlapped

Ans: B

Explanation: Overlap alone is allowed. Tj passes if Start(Tj) < Finish(Ti) < Validation(Tj) and write-set(Ti) does not intersect read-set(Tj).

Q66 | Read/Write Sets | Concurrency Controls

In validation, which sets are compared to detect a conflict between an earlier transaction Ti and a later transaction Tj?

A. only the write-sets of Ti and Tj

B. lock sets of Ti and Tj

C. write-set of Ti and read-set of Tj

D. read-set of Ti and read-set of Tj

Ans: C

Explanation: If Tj read something that Ti wrote, Tj may have missed Ti's update, so Tj must be aborted.

Q67 | Read Phase Effect | Concurrency Controls

During the read phase of optimistic scheduling, other transactions:

A. can read only committed versions, and only with an S-lock held

B. are not blocked, since no locks are held on the items

C. must wait until the read phase of that transaction ends

D. are rolled back at once if they touch the same item in any mode

Ans: B

Explanation: No locks are used in the read phase. Conflicts are handled later by validation.

Q68 | Not a Phase | Concurrency Controls

Which of the following is NOT a phase of optimistic scheduling?

A. Write phase

B. Lock acquisition phase

C. Read phase

D. Validation phase

Ans: B

Explanation: Optimistic scheduling is lock free; its phases are read, validation and write.

Section 5: MVT (Multiversion Timestamp Ordering)

Q69 | Multiversion Idea | Concurrency Controls

In a multiversion scheme, when a transaction writes a data item:

A. a new version of the item is created and older versions are retained

B. the old value is simply overwritten in place by the new value

C. the item is locked permanently against all further access

D. the write is delayed until all the readers of the item finish

Ans: A

Explanation: Multiple versions let readers use a version suited to their timestamp.

Q70 | MVT Advantage | Concurrency Controls

A key advantage of multiversion schemes is that:

A. storage requirement is reduced because old values are discarded

B. write operations never conflict with each other at all

C. reads are never rejected, since a suitable version exists

D. timestamps are not needed by the protocol for ordering

Ans: C

Explanation: A reader gets the proper older version instead of being rolled back. The price is extra storage.

Q71 | Version Contents | Concurrency Controls

In multiversion timestamp ordering, each version Qk stores:

A. only the value and lock mode

B. only the value and the database commit time

C. the value and a validation timestamp

D. the value, W-timestamp(Qk) and R-timestamp(Qk)

Ans: D

Explanation: W-ts is the creator's timestamp; R-ts is the largest timestamp of a transaction that read that version.

Q72 | Version W-ts | Concurrency Controls

The W-timestamp of a version Qk equals:

A. the timestamp of the oldest transaction that is still active

B. the timestamp of the transaction that created that version

C. the time of the last database checkpoint that was taken

D. the timestamp of the last transaction that read that version

Ans: B

Explanation: Each version is stamped with its writer's timestamp.

Q73 | Version Selection | Concurrency Controls

Item Q has versions with W-ts 4, 9 and 15. T with TS(T) = 12 issues read(Q). T reads the version with W-ts:

A. 4

B. 9

C. 15

D. 12

Ans: B

Explanation: Read uses the version with the largest W-ts not exceeding TS(T). Here that is 9, since 15 > 12.

Q74 | MV Write Allowed | Concurrency Controls

Q has versions with W-ts 5 and 12. The version with W-ts 5 has R-ts 8. T with TS(T) = 10 issues write(Q). Outcome?

A. Allowed; a new version with W-ts 10 is created

B. Rejected because 10 > 8

C. Allowed; the version with W-ts 5 is overwritten

D. Rejected because 10 < 12

Ans: A

Explanation: The version selected is W-ts 5. TS(T) = 10 is not less than its R-ts 8, so a new version is created.

Q75 | MV Write Rejected | Concurrency Controls

Q has versions with W-ts 5 and 12. The version with W-ts 5 has R-ts 11. T with TS(T) = 10 issues write(Q). Outcome?

A. T is rolled back

B. A new version with W-ts 10 is created

C. The version with W-ts 12 is overwritten

D. The write is ignored

Ans: A

Explanation: A reader with timestamp 11 already read version 5, but T (TS 10) would create a version it should have read. TS(T) < R-ts(Q5), so T aborts.

Q76 | MV Drawback | Concurrency Controls

A disadvantage of multiversion schemes is:

A. every write requires an exclusive lock on the whole database

B. reads may be rolled back frequently because of old version use

C. extra storage for old versions and the need to remove obsolete ones

D. serializability can no longer be guaranteed by the protocol

Ans: C

Explanation: Old versions pile up, so garbage collection of versions no longer needed is required.

Q77 | Read-Only Transactions | Concurrency Controls

In a multiversion scheme, read-only transactions:

A. must wait until all of the active writers have committed, and only then read

B. can read the uncommitted versions that are made by any transaction

C. see a consistent snapshot without blocking, or being blocked by, writers

D. must acquire X locks on all of the items they read

Ans: C

Explanation: Readers choose versions by timestamp, so they do not interfere with concurrent updates.

Q78 | Version Removal | Concurrency Controls

A version Qk can be safely deleted in a multiversion scheme when:

A. a newer version exists whose W-ts is not greater than the oldest active transaction's timestamp

B. its W-ts is the largest among all of the versions of the item Q

C. its R-ts is zero, so no transaction has ever read it at all

D. it was created by the oldest transaction that exists in the system now

Ans: A

Explanation: No active transaction can ever need the older version, since all of them would read the newer one instead.

Q79 | Which Operation Aborts | Concurrency Controls

In multiversion timestamp ordering, which operation may be rolled back because of a timestamp conflict?

A. A write operation only

B. Both reads and writes

C. Neither reads nor writes

D. A read operation only

Ans: A

Explanation: Reads are served from a suitable version and never fail. A write fails if TS(T) < R-ts of the version it would follow.

Q80 | MVT Cost-Benefit | Concurrency Controls

Compared with single-version timestamp ordering, a multiversion scheme needs more _____ but rejects fewer _____.

A. timestamps; write operations

B. storage; read operations

C. validation phases; read operations

D. memory for locks; write operations

Ans: B

Explanation: Keeping old versions costs storage, but reads are always served from a suitable version.

Q81 | MVT Expansion | Concurrency Controls

In concurrency control, MVT stands for:

A. Multiversion Timestamp Ordering

B. Multiple Validation Technique

C. Multi-Version Transaction Locking

D. Managed Version Tracking Table

Ans: A

Explanation: MVT keeps several versions of each item and orders operations by transaction timestamps.

Q82 | New Version Stamps | Concurrency Controls

When T executes a successful write(Q) in MVT, the new version Qk gets:

A. W-ts(Qk) = 0 and R-ts(Qk) = TS(T)

B. W-ts(Qk) = R-ts(Qk) = the largest timestamp in the system

C. W-ts(Qk) = R-ts(Qk) = TS(T)

D. W-ts(Qk) = TS(T) and R-ts(Qk) = 0

Ans: C

Explanation: The creator's timestamp is stored as both the write and the initial read timestamp of the new version.

Q83 | Read Updates R-ts | Concurrency Controls

T with TS(T) = 14 reads version Qk, whose R-ts(Qk) is 9. After the read, R-ts(Qk) is:

A. 23

B. 0

C. 9

D. 14

Ans: D

Explanation: R-ts(Qk) is raised to TS(T) when TS(T) is larger, so R-ts(Qk) = max(9, 14) = 14.

Q84 | R-ts Never Lowered | Concurrency Controls

T with TS(T) = 6 reads version Qk with W-ts 4 and R-ts 11. After the read, R-ts(Qk) is:

A. 11

B. 4

C. 6

D. 17

Ans: A

Explanation: R-ts(Qk) keeps the larger value, max(11, 6) = 11, so it never decreases.

Q85 | Why Write Is Rejected | Concurrency Controls

In MVT, T's write(Q) is rejected when TS(T) < R-ts(Qk) of the selected version because:

A. T does not hold an exclusive lock on the item

B. a younger transaction has already read that version, which T's new version would have replaced for it

C. the selected version is older than the database

D. T would overwrite the selected version in place

Ans: B

Explanation: The younger reader should have seen T's version but read Qk instead, so the timestamp order would be violated.

Section 6: Transaction Schedule-Based Questions

Q86 | Schedule - Lost Update | Concurrency Controls

Identify the problem in the following schedule (A is the same item):

T1	T2
Read(A)	
	Read(A)
Write(A)	
	Write(A)
Commit	
	Commit

A. Incorrect summary

B. Dirty read

C. Unrepeatable read

D. Lost update

Ans: D

Explanation: Both read the old A, and T2's write overwrites T1's. T1's update is lost.

Q87 | Schedule - Dirty Read | Concurrency Controls

Identify the problem in the following schedule:

T1	T2
Write(A)	
	Read(A)
Abort	
	Commit

A. Lost update

B. Unrepeatable read

C. Phantom read

D. Dirty read

Ans: D

Explanation: T2 read A written by T1 before T1 aborted. T2 used a value that was rolled back.

Q88 | Schedule - Unrepeatable | Concurrency Controls

Identify the problem in the following schedule:

T1	T2
Read(A)	
	Write(A)
	Commit
Read(A)	
Commit	

A. Dirty read

B. Unrepeatable read

C. Lost update

D. Phantom read

Ans: B

Explanation: T1 gets different values for A on its two reads because T2 committed an update in between.

Q89 | Schedule - Summary | Concurrency Controls

T1 computes A + B while T2 transfers an amount from A to B. Identify the problem:

T1	T2
Read(A)	
	Read(A); Write(A)
	Read(B); Write(B)
	Commit
Read(B)	
Commit	

A. Lost update (write-write overwrite)

B. Unrepeatable read (item changed)

C. Dirty read (aborted writer)

D. Incorrect summary (inconsistent retrieval)

Ans: D

Explanation: T1 read A before T2's update and B after it, so the sum mixes old and new values. T2 had committed before T1 read B, so it is not a dirty read.

Q90 | Schedule - Timestamp | Concurrency Controls

TS(T1) = 5 and TS(T2) = 10. Under basic timestamp ordering, what happens in this schedule?

T1	T2
	Read(A)
Write(A)	

A. Write(A) of T1 is ignored by the Thomas rule

B. Write(A) of T1 succeeds and W-ts(A) becomes 5

C. Write(A) of T1 is rejected and T1 is rolled back

D. Read(A) of T2 is rejected

Ans: C

Explanation: After T2's read, R-ts(A) = 10. T1's write has TS 5 < 10, so T1 is too late and is rolled back.

Q91 | Schedule - 2PL Sequence | Concurrency Controls

Which sequence of lock operations by a single transaction obeys 2PL?

A. Lock(A), Lock(B), Unlock(B), Lock(C), Unlock(A), Unlock(C)

B. Lock(A), Lock(B), Unlock(A), Unlock(B)

C. Lock(A), Unlock(A), Lock(A), Unlock(A)

D. Lock(A), Unlock(A), Lock(B), Unlock(B)

Ans: B

Explanation: All locks must be acquired before any is released. The other options lock again after an unlock.

Q92 | Schedule - Lock Grants | Concurrency Controls

Requests arrive in this order on item A. Which requests are granted immediately?

T1	T2	T3
Lock-S(A)		
	Lock-S(A)	
		Lock-X(A)

A. T1's and T3's requests; T2 waits

B. Only T1's request; T2 and T3 wait

C. All three requests at once

D. T1's and T2's S-locks only; T3 waits

Ans: D

Explanation: S-S is compatible, so T1 and T2 share A. T3's X-lock conflicts with both, so it waits until both release.

Q93 | Schedule - Optimistic | Concurrency Controls

T1 and T2 follow optimistic scheduling as below. What happens when T1 is validated?

T1	T2
Start; Read(A)	
	Start; Read(A)
	Validate; Write(A); Finish
Validate	

A. T1 succeeds, because T1 only reads A and never writes

B. T1 fails, simply because T1 started before T2 did

C. T1 succeeds, because T2 finished its write phase first

D. T1 fails; T2 wrote A, which T1 read, after T1 had started

Ans: D

Explanation: Finish(T2) > Start(T1) and write-set(T2) overlaps read-set(T1), so T2 cannot precede T1. T1 is aborted.

Q94 | Schedule - Multiversion | Concurrency Controls

In multiversion timestamp ordering with TS(T1) = 5 and TS(T2) = 10, what does T1 read in this schedule (A0 = initial version, W-ts 0)?

T1	T2
	Write(A)
Read(A)	

A. The initial version A0

B. The version written by T2 (W-ts 10)

C. T1 waits until T2 commits and then writes Q

D. Nothing; T1 is rolled back as 5 < 10

Ans: A

Explanation: T1 reads the version with the largest W-ts <= 5, which is A0. In basic TO this same read would be rejected.

Q95 | Schedule - Conflicts | Concurrency Controls

Consider S: R1(A), R2(A), W2(A), R1(B), W1(B). Which pair of operations conflicts?

A. R1(A) and R2(A)

B. R1(A) and W2(A)

C. W2(A) and W1(B)

D. R1(A) and R1(B)

Ans: B

Explanation: Operations conflict if they belong to different transactions, access the same item and at least one is a write.

Q96 | Schedule - Serializability | Concurrency Controls

The following schedule is:

T1	T2
Read(A)	
	Write(A)
	Read(B)
Write(B)	

A. a serial schedule

B. equivalent to the serial schedule T1 then T2

C. not conflict serializable

D. equivalent to the serial schedule T2 then T1

Ans: C

Explanation: R1(A) before W2(A) forces T1 before T2, but R2(B) before W1(B) forces T2 before T1. This cycle means it is not conflict serializable.

Q97 | Schedule - Recoverability | Concurrency Controls

Classify the following schedule:

T1	T2
Write(A)	
	Read(A)
	Commit
Abort	

A. Recoverable and cascadeless

B. A serial schedule

C. Recoverable but not cascadeless

D. Not recoverable

Ans: D

Explanation: T2 read T1's uncommitted write and committed before T1 aborted. T2's commit cannot be undone, so the schedule is not recoverable.

Q98 | Schedule - Cascade | Concurrency Controls

If T1 aborts in the schedule below, which transactions must also be rolled back?

T1	T2	T3
Write(A)		
	Read(A)	
	Write(B)	
		Read(B)
Abort		

A. T2 only

B. T2 and T3

C. None of them

D. T3 only

Ans: B

Explanation: T2 read T1's uncommitted value of A, and T3 read T2's uncommitted value of B, so both depend on T1.

Q99 | Schedule - Equivalent Serial | Concurrency Controls

The following schedule is:

T1	T2
Read(A); Write(A)	
	Read(A); Write(A)
Read(B); Write(B)	
	Read(B); Write(B)

A. a lost update schedule

B. conflict serializable, equivalent to T1 followed by T2

C. conflict serializable, equivalent to T2 followed by T1

D. not conflict serializable

Ans: B

Explanation: Every conflicting pair (on A and on B) has T1's operation first, so the only precedence is T1 to T2.

Q100 | Schedule - 2PL Violation | Concurrency Controls

T1 executes: Lock-X(A), Write(A), Unlock(A), Lock-X(B), Write(B), Unlock(B). This violates 2PL because:

A. T1 unlocks B at the very end of its own execution

B. T1 uses exclusive locks on the items for each of its writes

C. T1 locks A before it writes A, which is a normal step

D. T1 requests Lock-X(B) after releasing a lock on A

Ans: D

Explanation: In 2PL no lock may be acquired after the first unlock, and Unlock(A) happens before Lock-X(B).

Q101 | Schedule - TS Sequence | Concurrency Controls

TS(T1) = 10 and TS(T2) = 20; initially R-ts(Q) = W-ts(Q) = 0. Under basic TO, what happens to T1's write(Q) in this schedule?

T1	T2
Read(Q)	
	Write(Q)
Write(Q)	

A. T1 waits until T2 commits and then writes Q

B. T2 is rolled back instead, since T2 wrote last

C. T1 is rolled back, since TS(T1) is less than W-ts(Q) = 20

D. T1's write succeeds, and W-ts(Q) becomes 10 afterwards

Ans: C

Explanation: After T1's read R-ts(Q) = 10, and after T2's write W-ts(Q) = 20. T1's write is then too late (10 < 20).

Section 7: Comparisons and Tricky Questions

Q102 | Locking vs Timestamp | Concurrency Controls

When a conflict arises, lock-based protocols typically _____, whereas timestamp ordering typically _____.

A. ignore the operation; make a transaction wait

B. make a transaction wait; roll back a transaction

C. roll back a transaction; make a transaction wait

D. make a transaction wait; make a transaction wait

Ans: B

Explanation: Locks resolve conflicts by blocking. Timestamp ordering has no waiting and rolls back late transactions.

Q103 | Statement-Based | Concurrency Controls

Consider: I. Several transactions may hold S-locks on the same item. II. An X-lock lets several transactions read the item simultaneously. III. A dirty read involves reading uncommitted data. Which are correct?

A. I, II and III

B. I and II only

C. I and III only

D. II and III only

Ans: C

Explanation: X-locks are exclusive, so II is false. I and III are correct.

Q104 | Assertion-Reason | Concurrency Controls

Assertion (A): Optimistic scheduling is preferred when conflicts are rare.
Reason (R): It avoids locking overhead and checks conflicts only at validation.

A. Both A and R are true and R is the correct explanation of A

B. Both A and R are true but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: A

Explanation: Because conflicts are rare, deferring checks to validation avoids lock overhead with few aborts.

Q105 | Assertion-Reason | Concurrency Controls

Assertion (A): Basic 2PL guarantees conflict serializability.
Reason (R): In basic 2PL every transaction holds all its locks until it commits.

A. Both A and R are true and R is the correct explanation of A

B. Both A and R are true but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: C

Explanation: A is true, but R is false: holding all locks until commit is strict/rigorous 2PL. Basic 2PL only requires no lock after the first unlock.

Q106 | Assertion-Reason | Concurrency Controls

Assertion (A): In optimistic scheduling, a transaction in its read phase writes directly to the database.
Reason (R): Validation is performed before the write phase.

A. Both A and R are true and R is the correct explanation of A

B. Both A and R are true but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: D

Explanation: A is false: updates go to local copies during the read phase. R is true: validation precedes the write phase.

Q107 | Match the Following | Concurrency Controls

Match the concept with its description:
P. Shared lock Q. Thomas write rule R. Validation phase S. Version selection in MVT
1. Ignores an obsolete write 2. Checks conflicts before updating the database 3. Allows multiple readers 4. Uses the version with largest W-ts not exceeding TS(T)

A. P-1, Q-3, R-2, S-4

B. P-3, Q-1, R-2, S-4

C. P-3, Q-1, R-4, S-2

D. P-2, Q-1, R-3, S-4

Ans: B

Explanation: Shared lock = many readers (3); Thomas rule ignores obsolete writes (1); validation checks before write (2); version selection uses W-ts <= TS(T) (4).

Q108 | No Locks | Concurrency Controls

Which techniques do not use locks to control access while transactions execute?

A. Timestamp ordering and the basic 2PL protocol

B. Two-phase locking (2PL) and strict 2PL protocols

C. Timestamp ordering and optimistic scheduling

D. Optimistic scheduling and the strict 2PL protocol

Ans: C

Explanation: Timestamp ordering uses rollback, and optimistic scheduling uses validation. 2PL variants are lock based.

Q109 | Cheap Abort | Concurrency Controls

Aborting a transaction in optimistic scheduling does not require undoing database changes because:

A. updates stay in private copies until the write phase

B. locks preserve the old values until the transaction ends

C. every write is first stored as a timestamp on the item

D. the database is read-only during all of the three phases

Ans: A

Explanation: Nothing reaches the database before successful validation, so abort only discards local copies.

Q110 | Pessimistic Scheme | Concurrency Controls

Which scheme is called pessimistic?

A. Validation-based protocol

B. Optimistic scheduling

C. Snapshot read in multiversion schemes

D. Lock-based concurrency control

Ans: D

Explanation: Locking assumes conflicts are likely, so it prevents them in advance by locking. Optimistic schemes assume the opposite.

Q111 | Basic TO vs MVT | Concurrency Controls

A read(Q) by T with TS(T) < W-ts(Q) is rejected in basic TO. In a multiversion scheme the same read:

A. is served from the newest available version

B. is served from an older suitable version

C. waits for the writer to commit first

D. is also rejected and T is rolled back

Ans: B

Explanation: Multiversion schemes keep old versions, so the reader gets the version matching its timestamp.

Q112 | False Statement (MVT) | Concurrency Controls

Which statement about multiversion schemes is FALSE?

A. Writers overwrite the older version of the item in place

B. Each version carries a timestamp of its creator

C. Extra storage is needed for old versions

D. Reads need not be rejected because an older version can be used

Ans: A

Explanation: Multiversion schemes create a new version on each write instead of overwriting, so the statement is false.

Q113 | Detection Time | Concurrency Controls

In which phase does optimistic scheduling detect conflicts?

A. At start, when the timestamps are first assigned

B. Write phase, after updating the database items

C. Validation phase, at the end of the transaction

D. Read phase, at the time each item is accessed

Ans: C

Explanation: Locking and timestamp ordering detect conflicts at access time. Optimistic scheduling detects them only at validation.

Q114 | Serial Order Basis | Concurrency Controls

The equivalent serial order is decided by lock points in _____ and by timestamps in _____.

A. optimistic scheduling; 2PL

B. multiversion scheme; optimistic scheduling

C. 2PL; timestamp ordering

D. timestamp ordering; 2PL

Ans: C

Explanation: 2PL orders transactions by lock point and timestamp ordering by TS(T). The validation-based scheme uses Validation(T).

Q115 | Statement-Based (Schedules) | Concurrency Controls

Consider: I. Strict 2PL schedules are cascadeless. II. Basic timestamp ordering guarantees cascadeless schedules. III. Every recoverable schedule is cascadeless. IV. Every cascadeless schedule is recoverable. Which are correct?

A. II and IV only

B. II and III only

C. I and III only

D. I and IV only

Ans: D

Explanation: Cascadeless is a stricter property than recoverable, so IV is true and III is false. Basic TO allows cascading rollbacks (II false).

Q116 | False Statement (TS) | Concurrency Controls

Which statement about timestamp ordering is FALSE?

A. A transaction with a larger timestamp is treated as the older transaction

B. A rolled-back transaction restarts with a new, larger timestamp

C. Conflicts are resolved by rollback and without using any locks

D. R-ts(Q) never decreases, since it keeps the largest reader timestamp

Ans: A

Explanation: A smaller timestamp means an older transaction, which comes earlier in the equivalent serial order.

Q117 | Assertion-Reason | Concurrency Controls

Assertion (A): Every cascadeless schedule is serializable.
Reason (R): Cascadeless schedules prevent dirty reads.

A. Both A and R are true and R is the correct explanation of A

B. Both A and R are true but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: D

Explanation: A is false: a cascadeless schedule can still be non-serializable. R is true, since reads only follow commits.

Q118 | Detect and Resolve | Concurrency Controls

Compared with locking, optimistic scheduling detects conflicts _____ and resolves them by _____.

A. at the end of the transaction; aborting and restarting it

B. at each data access; making the transaction wait for a lock

C. at the start of the transaction; granting locks to it

D. at each data access; ignoring the write operation

Ans: A

Explanation: Locking detects conflicts at access time and blocks. Optimistic detects them at validation and restarts the transaction.

Section 8: Mixed OAVS TGT Practice Set

Q119 | Lock Upgrade (Tricky) | Concurrency Controls

T1 holds an S-lock on Q and is the only holder. T1 requests an upgrade to an X-lock. The lock manager will:

A. make T1 wait until it releases its own S-lock first

B. abort T1 and restart it with a new timestamp

C. refuse, because an S-lock can never be upgraded to an X-lock

D. grant the upgrade, since no other transaction holds a lock on Q

Ans: D

Explanation: An upgrade is granted when no other transaction holds a conflicting lock.

Q120 | Application - Booking | Concurrency Controls

Two users read seats_left = 1, both book and both write seats_left = 0, selling two seats. With locking, each booking transaction should:

A. read the record twice before it finally writes the new value back

B. obtain only an S-lock on the record, and then write to it directly

C. obtain an X-lock on the seat record before reading it for update

D. release its lock immediately after reading the record, then update it

Ans: C

Explanation: An X-lock before the read-update makes the second user wait, preventing the lost update.

Q121 | Application - Audit | Concurrency Controls

A bank runs long read-only audit queries while updates continue. Which approach lets audits run without blocking updates?

A. X-locks on every audited item

B. A multiversion approach with snapshot reads

C. Running audits serially before any update

D. Strict 2PL with S-locks on all audited items

Ans: B

Explanation: Readers use older versions, so they neither block nor are blocked by writers.

Q122 | Application - Catalogue | Concurrency Controls

An online catalogue is browsed by thousands of users and updated rarely. Which concurrency approach suits it best?

A. Strict 2PL with X-locks even for reads

B. Serial execution of all transactions

C. Exclusive locks on every item viewed

D. Optimistic scheduling

Ans: D

Explanation: Low conflict means optimistic validation rarely fails and no locking overhead is paid on reads.

Q123 | Assertion-Reason | Concurrency Controls

Assertion (A): Strict 2PL avoids cascading rollbacks.
Reason (R): Basic 2PL guarantees serializability.

A. Both A and R are true and R is the correct explanation of A

B. Both A and R are true but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: B

Explanation: Strict 2PL holds X-locks until commit, which stops reads of uncommitted data. R is true but does not explain A.

Q124 | Match the Following | Concurrency Controls

Match the problem with its description:
P. Lost update Q. Dirty read R. Incorrect summary S. Unrepeatable read
1. Aggregate sees partial updates of another transaction 2. Value changes between two reads of the same item
3. One transaction's write is overwritten 4. Reads uncommitted data later rolled back

A. P-3, Q-4, R-2, S-1

B. P-4, Q-3, R-1, S-2

C. P-3, Q-4, R-1, S-2

D. P-3, Q-1, R-4, S-2

Ans: C

Explanation: Lost update = overwritten (3); dirty read = uncommitted data (4); incorrect summary = partial updates (1); unrepeatable read = changed between reads (2).

Q125 | Assertion-Reason | Concurrency Controls

Assertion (A): A transaction rolled back under timestamp ordering is restarted with a new timestamp.
Reason (R): With its old timestamp it would be rejected again by the same conflict.

A. Both A and R are true and R is the correct explanation of A

B. Both A and R are true but R is not the correct explanation of A

C. A is true but R is false

D. A is false but R is true

Ans: A

Explanation: The old timestamp is still too small for the conflicting item, so a new, larger timestamp is needed.

Q126 | Statement-Based | Concurrency Controls

Consider: I. Strict 2PL releases all locks as early as possible. II. In optimistic scheduling, updates are applied only after successful validation. III. Multiversion schemes keep older versions of items. IV. The Thomas write rule rolls back T whenever TS(T) < W-ts(Q). Which are correct?

A. II and III only

B. I and II only

C. I and IV only

D. III and IV only

Ans: A

Explanation: Strict 2PL holds X-locks until commit (I false). The Thomas rule ignores an obsolete write instead of rolling back (IV false).

Q127 | Lock Timing (Tricky) | Concurrency Controls

X-locks are held until commit, but S-locks are released immediately after reading. Which problem is prevented?

A. Unrepeatable read

B. Incorrect summary

C. Lost update

D. Dirty read

Ans: D

Explanation: No one can read uncommitted writes. But early S-lock release still allows lost update, unrepeatable read and incorrect summary.

Q128 | Rigorous 2PL | Concurrency Controls

In rigorous 2PL, a transaction releases its locks:

A. only exclusive locks at commit; shared ones immediately

B. only at commit or abort, and this includes shared locks

C. as soon as it obtains its last lock

D. at the start of the validation phase

Ans: B

Explanation: Rigorous 2PL holds both S and X locks to the end, so the serial order matches commit order.

Q129 | Validation Rule | Concurrency Controls

In validation, Ti is validated before Tj and Finish(Ti) < Start(Tj). Then:

A. both transactions must be rolled back and restarted

B. Tj must be aborted and restarted by the system

C. no check is needed, since Ti finished before Tj started

D. Ti must be aborted and restarted by the system

Ans: C

Explanation: Ti completed fully before Tj began, so the serial order Ti then Tj is preserved.

Q130 | Starvation | Concurrency Controls

Repeated rollback and restart of the same transaction under timestamp ordering can cause:

A. a phantom row in a query result

B. a dirty read of uncommitted data

C. a lost update on the item it writes

D. starvation

Ans: D

Explanation: If a transaction keeps meeting conflicts with newer transactions, it may be restarted again and again without finishing.

Q131 | Hot Item Application | Concurrency Controls

Many concurrent transactions repeatedly update the same hot item. Which approach is likely to waste the most completed work?

A. Conservative 2PL, since locks are taken before execution

B. Multiversion snapshot reads, since readers never block

C. Strict 2PL, since writers queue for the X-lock

D. Optimistic scheduling, due to repeated validation failures

Ans: D

Explanation: Optimistic transactions do all their work and then abort on validation. Lock-based writers wait but do not lose finished work.

Q132 | Lock Point Order | Concurrency Controls

Under 2PL, T1 obtains its last lock at time 5 and T2 obtains its last lock at time 9. The equivalent serial order is:

A. T1 followed by T2

B. T1 and T2 simultaneously

C. T2 followed by T1

D. It cannot be determined

Ans: A

Explanation: 2PL schedules are equivalent to a serial schedule ordered by lock points.

Q133 | Repeatable Reads | Concurrency Controls

Under locking, an item read twice in a transaction returns the same value if the transaction:

A. validates its read-set at the start of the transaction

B. releases its S-lock on the item right after each of its reads

C. uses only X-locks for its writes and no S-locks for reads

D. holds its S-lock on the item until the end of the transaction

Ans: D

Explanation: Holding the S-lock stops other transactions from writing the item in between, which prevents unrepeatable reads.

Answer Key

Q1: A	Q2: C	Q3: D	Q4: C	Q5: A	Q6: C	Q7: B	Q8: C
Q9: B	Q10: B	Q11: C	Q12: B	Q13: D	Q14: D	Q15: A	Q16: B
Q17: D	Q18: B	Q19: A	Q20: A	Q21: C	Q22: B	Q23: C	Q24: A
Q25: A	Q26: B	Q27: D	Q28: C	Q29: B	Q30: B	Q31: D	Q32: A
Q33: A	Q34: C	Q35: A	Q36: B	Q37: D	Q38: B	Q39: A	Q40: C
Q41: D	Q42: B	Q43: C	Q44: B	Q45: D	Q46: C	Q47: A	Q48: D
Q49: A	Q50: A	Q51: C	Q52: D	Q53: A	Q54: A	Q55: C	Q56: C
Q57: B	Q58: C	Q59: D	Q60: D	Q61: C	Q62: A	Q63: A	Q64: D
Q65: B	Q66: C	Q67: B	Q68: B	Q69: A	Q70: C	Q71: D	Q72: B
Q73: B	Q74: A	Q75: A	Q76: C	Q77: C	Q78: A	Q79: A	Q80: B
Q81: A	Q82: C	Q83: D	Q84: A	Q85: B	Q86: D	Q87: D	Q88: B
Q89: D	Q90: C	Q91: B	Q92: D	Q93: D	Q94: A	Q95: B	Q96: C
Q97: D	Q98: B	Q99: B	Q100: D	Q101: C	Q102: B	Q103: C	Q104: A
Q105: C	Q106: D	Q107: B	Q108: C	Q109: A	Q110: D	Q111: B	Q112: A
Q113: C	Q114: C	Q115: D	Q116: A	Q117: D	Q118: A	Q119: D	Q120: C
Q121: B	Q122: D	Q123: B	Q124: C	Q125: A	Q126: A	Q127: D	Q128: B
Q129: C	Q130: D	Q131: D	Q132: A	Q133: D			